Decision Trees and Random Forests

Guided explanations, one source page at a time

Current coverage: all six source pages

Source: Statistical Learning: essentials — Trees and random forests, "Decision trees and random forests: A focused guide to the formulas used in applications A, B, and C," the six-page document provided for this discussion. This text expands all six pages for teaching; the numerical examples and conventions are retained from the source. Supplementary explanations and short derivations are explicitly pedagogical.

Contents

1 Source page 1: A tree is a sequence of questions **2**
1.1 What is a decision tree? . . . 2
1.2 Why does a regression leaf predict an average? . . . 2
1.3 How is the best numerical split chosen? . . . 3
1.4 Why are different child means useful? . . . 3
1.5 Efficient computation . . . 4

2 Source page 2: Classification — keep the leaf probabilities **5**
2.1 From numerical predictions to positive-class probabilities . . . 5
2.2 Gini impurity . . . 5
2.3 The weighted gain of a classification split . . . 6
2.4 Example: impurity improves while accuracy stays unchanged . . . 6
2.5 Categorical predictors and target leakage . . . 7

3 Source page 3: Control a tree's complexity **8**
3.1 Greedy construction: locally best does not imply globally best . . . 8
3.2 Why complexity must be controlled . . . 8
3.3 Cost-complexity pruning . . . 8
3.4 Exact pruning calculation from the six-row example . . . 9
3.5 Definition: Brier loss for binary classification . . . 10
3.6 Selecting complexity by cross-validation . . . 10

4 Source page 4: Bagging and random forests—average unstable trees **11**
4.1 Why put many trees together? . . . 11
4.2 Bootstrap sampling and aggregation . . . 11
4.3 Averaging probabilities is not majority voting . . . 12
4.4 Why does averaging help? The variance formula . . . 12
4.5 Random forests: reduce similarity between trees . . . 13
4.6 Which hyperparameter does what? . . . 13
4.7 Two different meanings of tree correlation . . . 13

5 Source page 5: OOB evaluation, importance, and decisions **15**
5.1 Out-of-bag observations arise automatically from bootstrap sampling . . . 15
5.2 How to calculate OOB predictions . . . 15
5.3 Feature importance I: mean decrease in impurity (MDI) . . . 16
5.4 Feature importance II: validation permutation importance . . . 16
5.5 Evaluate probabilities separately from decisions . . . 17
5.6 Choose a decision threshold from the error costs . . . 17
5.7 Three concepts that must remain separate . . . 18

6 Source page 6: The three companion notebooks **19**
6.1 A. A service with regime changes 19
6.2 B. Carseats: categorical features and probabilities 20
6.3 C. Bagging, decorrelation, OOB and importance 21
6.4 How the three applications fit together 22

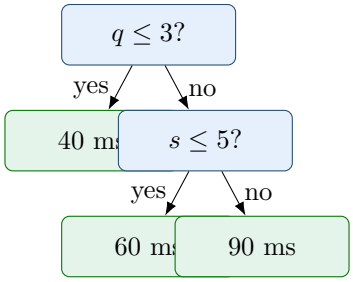
1 Source page 1: A tree is a sequence of questions

1.1 What is a decision tree?

A decision tree makes a prediction by successively asking questions about the input features. A *decision node* tests a condition, such as whether a queue has more than three waiting requests. The input follows one branch of each test until it reaches a *leaf*; the leaf stores a prediction.

The running example uses queue length q and payload size s (MB) to predict a service response time, in milliseconds:

$$f(s,q) = \begin{cases} 40, & q \le 3, \\ 60, & q > 3 \text{ and } s \le 5, \\ 90, & q > 3 \text{ and } s > 5. \end{cases} \tag{1}$$



Interaction without a product feature. The payload influences the prediction only if $q > 3$. A tree represents this conditional dependence directly, without having to introduce a feature such as $q\,s$ or a hand-crafted indicator. For ordinary numerical thresholds the leaves correspond to axis-aligned regions of feature space. The prediction is constant within each region; a standard tree also does not smoothly extrapolate beyond the range it has learned.

Vocabulary: The *depth* of a tree is the number of tests on its longest root-to-leaf path. A tree consisting of one leaf has depth zero. A *stump* has one split, hence depth one and two leaves. The displayed tree has depth two.

1.2 Why does a regression leaf predict an average?

Suppose training observations indexed by a set A arrive at the same leaf. All must receive the same numerical prediction c. Least squares chooses c to minimize

$$E_A(c) = \sum_{i \in A} (y_i - c)^2. \tag{2}$$

Differentiating,

$$E'_A(c) = 2|A|c - 2\sum_{i \in A} y_i.$$

Since $E''_A(c) = 2|A| > 0$, the unique minimizer is

$$\boxed{c_A = \bar{y}_A = \frac{1}{|A|} \sum_{i \in A} y_i.} \tag{3}$$

The residual error in that leaf is

$$\boxed{\mathrm{RSS}(A) = \sum_{i \in A} (y_i - \bar{y}_A)^2.} \tag{4}$$

Thus a regression tree is a piecewise-constant model whose values are local training averages.

1.3 How is the best numerical split chosen?

Consider the six observations from the source:

x_i	1	2	3	4	5	6
y_i	2	3	2	8	9	10

The unsplit mean is $\bar{y} = 34/6 = 17/3$. The unsplit error is

$$\mathrm{RSS}(\text{parent}) = \sum_{i=1}^{6} (y_i - 17/3)^2 = \frac{208}{3}. \tag{5}$$

For a candidate threshold t, form the children

$$A_L = \{i : x_i \le t\}, \qquad A_R = \{i : x_i > t\},$$

and minimize $\mathrm{RSS}(A_L) + \mathrm{RSS}(A_R)$. Empty children or children below the permitted minimum size are not admissible.

For $t = 3.5$, the left responses are $(2, 3, 2)$ and the right responses are $(8, 9, 10)$. Their predictions and errors are

$$\bar{y}_L = 7/3, \qquad \mathrm{RSS}(A_L) = 2/3,$$
$$\bar{y}_R = 9, \qquad \mathrm{RSS}(A_R) = 2.$$

The total child error is therefore $8/3 \approx 2.67$. The source compares all candidate thresholds:

t	1.5	2.5	3.5	4.5	5.5
$\mathrm{RSS}(A_L) + \mathrm{RSS}(A_R)$	53.2	39.25	**8/3**	25.25	46.8

The winning test is $x \le 3.5$. It gives training root mean squared error

$$\mathrm{RMSE}_{\text{train}} = \sqrt{\frac{8/3}{6}} = \frac{2}{3}.$$

1.4 Why are different child means useful?

An important identity, stated on page 1, is

$$\boxed{\mathrm{RSS}(A) - \mathrm{RSS}(A_L) - \mathrm{RSS}(A_R) = \frac{n_L n_R}{n_L + n_R} (\bar{y}_L - \bar{y}_R)^2,} \tag{6}$$

where $n_L = |A_L|$ and $n_R = |A_R|$.

To see why it is true, write $\bar{y}$ for the parent average. On the left,

$$\sum_{i \in A_L} (y_i - \bar{y})^2 = \mathrm{RSS}(A_L) + n_L (\bar{y}_L - \bar{y})^2,$$

since the cross term sums to zero; the analogous identity holds on the right. Now $\bar{y} = (n_L\bar{y}_L + n_R\bar{y}_R)/(n_L + n_R)$. Substituting yields (6). In the six-row example,

$$\text{RSS reduction} = \frac{3 \cdot 3}{6}\left(9 - \frac{7}{3}\right)^2 = \frac{200}{3}.$$

Interpretation: A useful split separates regions with significantly different mean responses, taking the sizes of both groups into account.

1.5 Efficient computation

For each numerical feature, sort the rows by that feature and check midpoints between *distinct* feature values: thresholds within the same gap produce identical partitions. For sorted responses $y_{(1)}, \ldots, y_{(n)}$, define prefix sums

$$S_k = \sum_{i=1}^{k} y_{(i)}, \qquad Q_k = \sum_{i=1}^{k} y_{(i)}^2.$$

The left-child RSS can be obtained immediately:

$$\boxed{\text{RSS}(\{1, \ldots, k\}) = Q_k - \frac{S_k^2}{k}.} \tag{7}$$

The right-child RSS has the same formula using the total sums minus their left prefixes. Sorting takes $O(n \log n)$ and then the scan is $O(n)$ per feature. When responses are close to a large common offset, centering them first improves numerical stability by reducing cancellation.

Remember. A low *training* RSS records success in fitting the observed data; it is not yet an estimate of prediction quality on unseen rows.

2 Source page 2: Classification — keep the leaf probabilities

2.1 From numerical predictions to positive-class probabilities

For binary classification the target is $y_i \in \{0, 1\}$. The quantity stored in a leaf is the observed proportion of positives,

$$\boxed{\hat{\pi}_A = \frac{\#\{i \in A : y_i = 1\}}{|A|}.} \tag{8}$$

For example, a leaf containing 20 customers of whom six defaulted on a loan predicts $\hat{\pi}_A = 6/20 = 0.30$. That is a probability estimate, not yet a binary decision. Once a classification threshold t is specified, the rule is

$$\hat{y} = \begin{cases} 1, & \hat{\pi}_A \geq t, \\ 0, & \hat{\pi}_A < t. \end{cases} \tag{9}$$

Equality is assigned to class 1, as in the source. The threshold is a *decision* choice and is not itself the criterion for selecting the tree's splits.

Caution on small leaves. If a leaf has only one observation and that observation is positive, (8) reports $\hat{\pi}_A = 1$. This does not establish near-certainty for future examples. A minimum leaf-size constraint can help reduce the instability of local frequency estimates.

2.2 Gini impurity

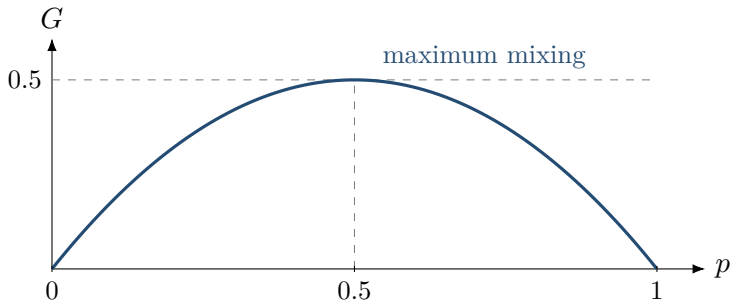
To evaluate splits for classification, the tree measures how mixed each node is. If a node contains K classes with proportions $p_1, \ldots, p_K$, its *Gini impurity* is

$$\boxed{G(A) = 1 - \sum_{k=1}^{K} p_k^2.} \tag{10}$$

In the binary case, write p for the positive-class proportion and $1 - p$ for the other class:

$$\boxed{G(A) = 1 - p^2 - (1-p)^2 = 2p(1-p).} \tag{11}$$

p	$G = 2p(1-p)$	Interpretation
0	0	pure negative node
0.2	0.32	mostly negative
0.5	0.50	maximum mixing
0.8	0.32	mostly positive
1	0	pure positive node



An exact probabilistic interpretation is worth noting: draw two class labels *independently* from the empirical distribution at the node. The probability that the labels differ is $1 - \sum_k p_k^2$, exactly the Gini impurity.

The source also presents *entropy* as an alternative impurity criterion:

$$\boxed{H(A) = -\sum_{k=1}^{K} p_k \log p_k, \qquad 0 \log 0 := 0.} \tag{12}$$

It likewise vanishes for a pure node and rewards splits that produce purer groups. Entropy and Gini need not rank every possible split identically.

2.3 The weighted gain of a classification split

Write I for the chosen impurity, either G or H. If node A of size n_A is partitioned into children of sizes n_L, n_R, the gain is

$$\boxed{\text{gain} = I(A) - \frac{n_L}{n_A} I(A_L) - \frac{n_R}{n_A} I(A_R).} \tag{13}$$

A classification tree chooses the admissible split with the largest gain. The weights are essential: otherwise a tiny pure child could exert as much influence as a child containing almost every observation.

For comparison with page 1, if in regression one defines the node impurity as $I(A) = \mathrm{RSS}(A)/n_A$, then the weighted gain equals

$$\frac{\mathrm{RSS}(A) - \mathrm{RSS}(A_L) - \mathrm{RSS}(A_R)}{n_A}.$$

Hence maximizing the gain gives exactly the same split ranking as minimizing the total child RSS.

2.4 Example: impurity improves while accuracy stays unchanged

Page 2 gives a parent with eight positive and two negative rows. A proposed split creates a pure left child of five positives and a right child containing three positives and two negatives:

Node	Positives	Negatives	$\hat{\pi}$	Gini
Parent	8	2	0.8	0.32
Left child	5	0	1.0	0
Right child	3	2	0.6	0.48

The weighted child impurity is

$$\frac{5}{10} \cdot 0 + \frac{5}{10} \cdot 0.48 = 0.24,$$

so the Gini gain is

$$\boxed{0.32 - 0.24 = 0.08.}$$

This is a genuine improvement in impurity. However, at decision threshold $t = 0.5$, the unsplit parent predicts class 1 and *both* children still predict class 1. The hard predictions are unchanged: eight correct decisions, two wrong decisions, and accuracy $8/10 = 80\%$ both before and after the split.

Central lesson: Split quality in a classification tree is not equivalent to immediate accuracy gain. The split can distinguish regions with different estimated probabilities (1 versus 0.6) even if the two regions still have the same most likely class.

2.5 Categorical predictors and target leakage

Page 2 uses the *Carseats* data as an application. "Shelf location" is a nominal category. One-hot encoding creates indicators such as `ShelveLoc_Good`, permitting a split "Good" versus "not Good". Encoding nominal categories with arbitrary integers may accidentally impose an ordering that the categories do not possess. The supplied notebooks fit the encoder only on fitting data and represent an unseen category with an all-zero indicator vector; this handling rule must be explicit.

The binary target in the Carseats application is

$$\boxed{Y = \mathbf{1}_{\{\text{Sales}>8\}}.}$$

The original numerical variable `Sales` must *not* also be an input feature: it defines the answer the model is asked to predict. Including it would be *target leakage*. Similarly, a measurement available only after the desired prediction time is not a valid advance predictor.

Remember. Keep the leaf's positive-class probability for probability metrics and decisions involving unequal costs. Replacing it immediately by a hard class label throws away information about the degree of certainty.

3 Source page 3: Control a tree's complexity

3.1 Greedy construction: locally best does not imply globally best

A standard decision tree is built recursively. At each node, the algorithm finds an admissible split minimizing the sum of the child RSS values (regression) or maximizing the reduction in weighted impurity (classification). It then applies the same procedure separately to the children.

This is a *greedy* procedure: once a test has been selected, later splits do not cause it to be reconsidered. An initially weak split might permit substantially better subdivisions later, but the greedy algorithm does not search globally over all possible trees. Consequently, it need not find the best tree of a given size or depth. It is also *unstable*: a small change to training rows can change an early split and hence many subsequent leaves.

3.2 Why complexity must be controlled

A complex tree can have very low training error simply by dividing the data into many small, homogeneous leaves. In regression, a leaf with one training row has RSS zero; in binary classification it has Gini impurity zero. Yet such a leaf estimates the response from almost no evidence. A tree can therefore fit the noise in training observations rather than the underlying relationship. This is *overfitting*.

Page 3 highlights two direct complexity controls:

- `max_depth`: limits the number of tests along any root-to-leaf path; a smaller value produces a shallower tree.
- `min_samples_leaf`: requires every leaf to contain a minimum number of observations; increasing it forces predictions to rely on more data and can reduce variability.

These controls typically increase training error while potentially improving performance on unseen data. The best setting should be chosen by evaluation, not by the smallest training error.

Why scaling is usually unnecessary. If a numerical feature is transformed by a strictly increasing function, its ordering is unchanged. The tree can therefore form the same partitions by relabeling thresholds. This explains why ordinary numerical decision trees usually do not require standardization. Page 3 qualifies this invariance as holding in exact arithmetic; feature encoding, availability, and numerical implementation still matter.

3.3 Cost-complexity pruning

One approach grows a relatively large tree and then compares simpler *subtrees* obtained by removing branches. The document uses the penalized training objective

$$\boxed{R_\alpha(T) = R(T) + \alpha L(T), \qquad \alpha \geq 0,} \tag{14}$$

where $L(T)$ is the number of leaves. The term $\alpha L(T)$ places a cost on complexity. Small α emphasizes fit; large α favors fewer leaves.

The loss $R(T)$ depends on the prediction problem. For regression, it is normalized total RSS,

$$R(T) = \frac{1}{n} \sum_{\ell \text{ leaf}} \text{RSS}(A_\ell). \tag{15}$$

For Gini-based classification, it is the size-weighted average leaf impurity,

$$R(T) = \sum_{\ell \text{ leaf}} \frac{n_\ell}{n} G(A_\ell). \tag{16}$$

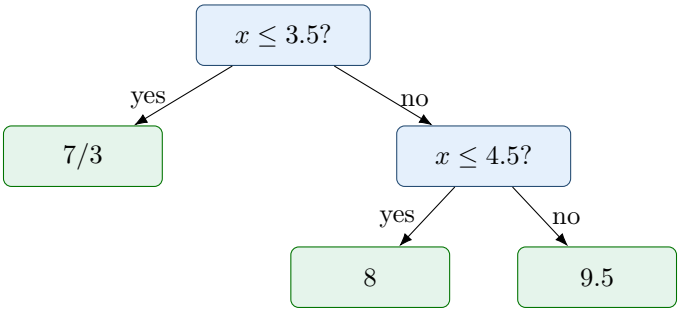
These have different units and numerical scales, so numerical α values are not directly comparable across regression and classification. If one used *unnormalized* RSS in place of normalized RSS for regression, the corresponding equivalent penalty would also be multiplied by n.

3.4 Exact pruning calculation from the six-row example

Return to the source-page-1 example:

$$x = (1, 2, 3, 4, 5, 6), \qquad y = (2, 3, 2, 8, 9, 10).$$

The first split is at $x = 3.5$, creating left responses $(2, 3, 2)$ and right responses $(8, 9, 10)$. A second test at $x = 4.5$ in the *right* child separates (8) from $(9, 10)$:



The three competing subtrees have the losses reported on source page 3:

Subtree	$L(T)$	$R(T)$	$R_\alpha(T)$
Keep both tests	3	7/36	$7/36 + 3\alpha$
Keep root test only	2	4/9	$4/9 + 2\alpha$
No tests	1	104/9	$104/9 + \alpha$

Here is where these errors come from:

$$\text{Two tests:} \quad \text{RSS} = (2/3) + 0 + (1/2) = 7/6, \qquad R = \frac{7/6}{6} = \frac{7}{36},$$

$$\text{Root only:} \quad \text{RSS} = (2/3) + 2 = 8/3, \qquad R = \frac{8/3}{6} = \frac{4}{9},$$

$$\text{No tests:} \quad \text{RSS} = 208/3, \qquad R = \frac{208/3}{6} = \frac{104}{9}.$$

First transition. Keeping both tests ties with keeping only the root when

$$\frac{7}{36} + 3\alpha = \frac{4}{9} + 2\alpha \quad \Longleftrightarrow \quad \boxed{\alpha = \frac{1}{4}}.$$

Second transition. Keeping the root test ties with eliminating all splits when

$$\frac{4}{9} + 2\alpha = \frac{104}{9} + \alpha \quad \Longleftrightarrow \quad \boxed{\alpha = \frac{100}{9}}.$$

Among these three subtrees, and choosing the smaller tree in case of a tie,

Penalty range	Selected tree
$0 \leq \alpha < 1/4$	3 leaves
$1/4 \leq \alpha < 100/9$	2 leaves
$\alpha \geq 100/9$	1 leaf

The switching points are properties of the *training* objective for these subtrees. They do not identify the penalty with the lowest error on future observations.

3.5 Definition: Brier loss for binary classification

Before using Brier score to choose a classification model, define the *Brier loss* of a single probability forecast. For an observed binary label $y \in \{0, 1\}$ and a forecast probability $p \in [0, 1]$ of class 1,

$$\boxed{\ell_{\text{Brier}}(y, p) = (y - p)^2.} \tag{17}$$

For m observations, its average—the *Brier score*—is

$$\boxed{\text{Brier}(y, \widehat{p}) = \frac{1}{m} \sum_{i=1}^{m} \ell_{\text{Brier}}(y_i, \widehat{p}_i) = \frac{1}{m} \sum_{i=1}^{m} (y_i - \widehat{p}_i)^2.} \tag{18}$$

Here $\widehat{p}_i$ is the estimated probability that observation i belongs to class 1, *not* the hard prediction obtained after thresholding. Smaller values are better; for binary outcomes the score lies between 0 (all forecasts perfectly correct) and 1 (all forecasts maximally wrong). For example, if $y = 1$ and $p = 0.8$, the loss is $(1 - 0.8)^2 = 0.04$; if $y = 0$ and $p = 0.8$, it is $(0 - 0.8)^2 = 0.64$. Thus confident wrong probabilities are penalized more heavily. This criterion measures probability quality and does not itself choose a classification threshold.

3.6 Selecting complexity by cross-validation

The page specifies a train/validation/test protocol, suitable for the independent-row examples in the companion notebooks:

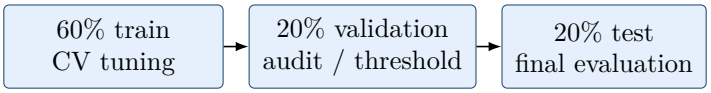
1. **Split before target exploration.** Partition the rows into 60% training, 20% validation and 20% test. For independent-data binary classification, stratify to preserve class proportions.

2. **Tune inside training only.** Define a parameter grid and use five-fold cross-validation on the 60% training partition. Each fold must fit a *fresh tree* and refit learned preprocessing only on its own fitting rows. Select by RMSE for regression or by Brier score when predicting binary probabilities.

3. **Refit the chosen configuration.** Using the selected hyperparameters, refit the model on the entire training partition.

4. **Audit on validation.** Inspect prediction quality, probabilities or decision thresholds, and potentially exploratory feature importance. If this analysis changes the decision policy, finalize it before testing.

5. **Evaluate once on held-out test.** Freeze the model and decision policy; estimate their performance with the previously unused test partition.



Why folds must be isolated. Growing a single tree on the full dataset *before* cross-validation and reusing its splits leaks information from the nominal validation fold into model fitting. The same issue applies to preprocessing parameters estimated from rows outside a fold.

Finally, the source emphasizes that random independent splits are not always appropriate: repeated observations of the same customer may require *grouped* splits; prediction of future observations may require *time-ordered* splits.

Remember. Pruning alters the fitted tree; cross-validation selects its complexity setting; and a separate held-out test estimates the predictive performance of the completed, frozen procedure.

4 Source page 4: Bagging and random forests—average unstable trees

4.1 Why put many trees together?

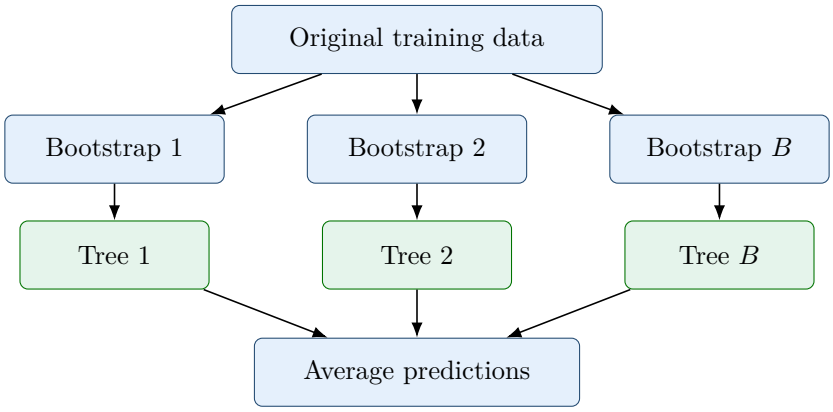
The preceding page explained two potential problems of decision trees: they can become very sensitive to the training observations, and their greedy splits can change sharply when those observations change. The aim of *bagging* (*bootstrap aggregating*) is to average several such unstable predictors, reducing their variability. This page distinguishes the construction of the ensemble from the mathematical reason averaging helps.

4.2 Bootstrap sampling and aggregation

Start with n training observations $D = \{(x_i, y_i) : i = 1, \ldots, n\}$. For each $b = 1, \ldots, B$:

1. Draw n indices independently and uniformly from $\{1, \ldots, n\}$ *with replacement*.
2. Fit a decision tree $\widehat{f}_b$ to the selected rows, counting repeated observations as often as they appear.
3. Retain the resulting prediction rule.

For example, drawing $(1, 3, 3, 4, 6, 6)$ from a six-row dataset is a legitimate bootstrap sample: rows 3 and 6 appear twice, while rows 2 and 5 are absent. Different samples tend to produce different splits and therefore different prediction rules.



Regression. If tree b outputs the number $\widehat{f}_b(x)$ at input x, the bagged prediction is the average

$$\boxed{\widehat{f}_B(x) = \frac{1}{B} \sum_{b=1}^{B} \widehat{f}_b(x).} \tag{19}$$

For instance, predictions of 42, 48 and 45 ms average to 45 ms.

Binary classification. If $\widehat{\pi}_b(x)$ is tree b's estimated *positive-class probability*, the source uses

$$\boxed{\widehat{\pi}_B(x) = \frac{1}{B} \sum_{b=1}^{B} \widehat{\pi}_b(x).} \tag{20}$$

When using software outputs, check that every probability refers to the same class: column order in probability arrays matters.

4.3 Averaging probabilities is not majority voting

Source page 4 gives the five class-1 probability predictions

$$0.2, \qquad 0.3, \qquad 0.4, \qquad 0.9, \qquad 0.95.$$

Their arithmetic mean is

$$\widehat{\pi}_B(x) = \frac{0.2 + 0.3 + 0.4 + 0.9 + 0.95}{5} = 0.55.$$

Under a 0.5 threshold, *averaging probabilities and then thresholding* gives class 1. By contrast, *thresholding each probability first and then taking a majority vote* gives three votes for class 0 and two for class 1, hence class 0.

Combination rule	Result	Classification ($t = 0.5$)
Mean of probabilities	0.55	1
Majority of hard labels	three 0s, two 1s	0

The first procedure preserves the magnitude of each probability estimate; the second discards that information before aggregating. Implementations should state which rule they use.

4.4 Why does averaging help? The variance formula

Fix an input x. Now regard the tree predictions $Z_b = \widehat{f}_b(x)$ as random quantities across *repeated training experiments*, including the sampling and randomization used to fit the trees. The source assumes, for this calculation, that every Z_b has variance σ_T^2 and that the pairwise correlation between any two distinct predictions is ρ. Hence

$$\mathrm{Cov}(Z_b, Z_c) = \rho\sigma_T^2 \qquad (b \neq c).$$

For the average $\overline{Z} = B^{-1}\sum_b Z_b$, expanding the variance gives

$$\begin{aligned}
\mathrm{Var}(\overline{Z}) &= \frac{1}{B^2}\left[\sum_{b=1}^{B}\mathrm{Var}(Z_b) + 2\sum_{b<c}\mathrm{Cov}(Z_b, Z_c)\right] \\
&= \frac{1}{B^2}\left[B\sigma_T^2 + B(B-1)\rho\sigma_T^2\right] \\
&= \boxed{\sigma_T^2\left(\rho + \frac{1-\rho}{B}\right).}
\end{aligned}$$

Here B diagonal variance contributions and $B(B-1)$ ordered cross-covariance contributions appear in the sum. This identity is the central mathematical point of the page.

Interpretation. When $\rho = 0$ (uncorrelated predictions), $\mathrm{Var}(\overline{Z}) = \sigma_T^2/B$; averaging becomes very effective as B grows. When $\rho = 1$ (perfectly correlated predictions), $\mathrm{Var}(\overline{Z}) = \sigma_T^2$: no variance is removed. For fixed positive correlation,

$$\lim_{B\to\infty}\mathrm{Var}(\overline{Z}) = \rho\sigma_T^2.$$

Thus averaging removes the *unshared* component of variability but cannot eliminate the part shared by all trees.

The numerical example in the source. Take $\sigma_T^2 = 16$ and $\rho = 0.25$. The formula gives

$$\mathrm{Var}(\overline{Z}) = 16\left(0.25 + \frac{0.75}{B}\right) = 4 + \frac{12}{B}.$$

Number of trees B	Variance of the average
1	16
10	5.2
100	4.12
$B \to \infty$	4

The first trees can greatly reduce variance, whereas increasing B from 100 to an even larger value yields diminishing returns. The residual variance 4 is not reduced simply by adding more correlated trees.

4.5 Random forests: reduce similarity between trees

Bootstrap sampling creates some variation between fitted trees, but trees may remain highly similar. For instance, when one feature is an especially strong predictor, many bootstrap trees may choose that same feature near the root. Their errors can then be correlated.

The random-forest modification described on page 4 works *at each splitting node*. If there are p candidate input features, instead of considering all p when finding the best split, draw a random subset of m features and search only within that subset. A fresh random subset is sampled at *every node*, not just once per tree.

Method	Randomness used	Features tested at each node
Bagging of trees	Bootstrap training rows	All p features
Random forest	Bootstrap rows and random node-level feature subsets	m of p features

With $m = p$, a bootstrap random forest behaves as bagging, assuming the other tree settings are equal. With $m < p$, different trees have more opportunities to find different splits. Their prediction correlation may fall, strengthening the effect of averaging. However, restricting candidate features can weaken individual trees, so there is a tradeoff; m must be selected by predictive evaluation, not by assuming smaller always means better.

4.6 Which hyperparameter does what?

Control	Primary role
B (number of trees)	Stabilizes ensemble averages; increases runtime and storage.
m (features offered per node)	Changes split options and often tree-to-tree similarity.
Maximum depth	Limits tree complexity; affects bias and variance.
Minimum leaf size	Controls averaging within leaves; affects bias and variance.

Averaging cannot correct fundamental problems such as target leakage, missing informative inputs, an unsuitable target, or irreducible outcome noise. Bagging also does not guarantee that bias disappears: a large number of similarly biased trees may still be biased.

4.7 Two different meanings of tree correlation

The source ends with a methodological warning. The correlation ρ in the variance formula concerns *predictions at one fixed input x* across repeated training experiments. In contrast, one

can take two fixed trained trees and calculate the correlation of their predictions *over many different inputs*. This second number measures similarity in how their prediction functions vary across the input space, but it is not the same quantity as the fixed-input repeated-training correlation ρ. Application C uses a prediction-vector similarity measure and separately resamples training data to investigate variability at fixed inputs.

Takeaway. Randomization makes diverse trees possible; averaging uses that diversity to make the final prediction more stable. Bagging injects diversity through bootstrap samples, while random forests add node-level feature randomization. Whether the resulting improvement is large for a given application must be evaluated on appropriate data.

5 Source page 5: OOB evaluation, importance, and decisions

This page covers three distinct questions about a fitted ensemble:

1. Can we assess its predictions without reserving additional observations from the training partition? This motivates *out-of-bag* evaluation.

2. Which input features does the model rely on? This motivates impurity-based and permutation feature importance.

3. How should estimated class probabilities be scored and converted to decisions when mistakes have different costs?

The three questions should not be conflated: estimating prediction quality, interpreting feature reliance, and choosing actions are separate activities.

5.1 Out-of-bag observations arise automatically from bootstrap sampling

In bagging, each tree is trained on n draws, with replacement, from n training rows. Fix one particular row i. On any one draw the probability that it is *not* selected equals $1 - 1/n$. The draws are independent, so its probability of being absent from the entire bootstrap sample is

$$\boxed{q_n = \left(1 - \frac{1}{n}\right)^n \longrightarrow e^{-1} \approx 0.368.} \tag{21}$$

For sufficiently large n, about 36.8% of the *original rows* are absent from any particular bootstrap sample, on average. Another way to say this is that a bootstrap sample of n entries usually contains about 63.2% distinct rows. Its length is still n: the remaining entries are repeats.

For B independently constructed bootstrap trees, the expected number of trees that exclude row i is Bq_n. For instance, with $B = 100$ and large n, the expected number is about 36.8. These are the only trees we may use for row i's OOB prediction.

5.2 How to calculate OOB predictions

Let

$$\mathcal{B}_i = \{b \in \{1, \ldots, B\} : \text{row } i \text{ was not selected for tree } b\}.$$

For binary classification, if tree b predicts positive-class probability $\widehat{\pi}_b(x_i)$, then

$$\boxed{\widehat{\pi}_i^{\text{OOB}} = \frac{1}{|\mathcal{B}_i|} \sum_{b \in \mathcal{B}_i} \widehat{\pi}_b(x_i), \qquad |\mathcal{B}_i| > 0.} \tag{22}$$

For regression, replace $\widehat{\pi}_b$ with the tree's numerical prediction $\widehat{f}_b$. After producing OOB predictions for the training rows, compare them with their observed targets using an appropriate evaluation metric.

Small illustrative example. Suppose five trees are fitted. Row i was included in the bootstrap samples for trees 1 and 4, but excluded from samples for trees 2, 3, and 5. Their class-1 probability predictions are 0.20, 0.50, and 0.80. The OOB prediction is

$$\widehat{\pi}_i^{\text{OOB}} = \frac{0.20 + 0.50 + 0.80}{3} = 0.50.$$

Trees 1 and 4 are *not used* in this calculation, regardless of what they predict for row i.

Coverage and limitations. It is possible that $|\mathcal{B}_i| = 0$ when the forest has only a few trees. Then the OOB prediction is *undefined*. Do not put zero in its place: exclude the row from the current OOB metric and report the fraction of rows having a defined OOB prediction (*coverage*). Both coverage and probabilities can be unstable with very small B.

OOB is an *internal* performance assessment, not an independent final test after extensive tuning on OOB metrics. Moreover, preprocessing estimated from the *entire* training dataset, rather than within the appropriate fitting subsets, can undermine the out-of-sample interpretation of OOB predictions. Keep a separate final test set for the frozen evaluation procedure.

5.3 Feature importance I: mean decrease in impurity (MDI)

A fitted classification tree associates each selected split with a reduction in impurity. If a node A splits into A_L and A_R, with sizes n_A, n_L, n_R, the *local impurity gain* is

$$\Delta I(A) = I(A) - \frac{n_L}{n_A} I(A_L) - \frac{n_R}{n_A} I(A_R).$$

Recall page 2: I can mean Gini impurity or entropy. In the mean-decrease-in-impurity approach, we add the gains of all splits that use a given feature, weighted by the fraction of training mass reaching each node. The scores are aggregated across trees and then normalized across features. Schematically, for feature j,

$$M_j = \sum_{\substack{\text{nodes } A \\ \text{splitting on } j}} \frac{n_A}{n_{\text{root}}} \Delta I(A), \qquad \text{MDI}_j = \frac{M_j}{\sum_k M_k}, \tag{23}$$

with the appropriate aggregation over trees understood. Normalization presupposes nonzero total impurity reduction.

What does MDI measure? It credits features that were *used by the fitted tree* to improve *training* node purity. This is not itself a measurement of held-out predictive performance and certainly not proof of causal influence. Features with many candidate thresholds can obtain artificially large scores; training overfit can further distort the interpretation.

5.4 Feature importance II: validation permutation importance

Permutation importance asks a more directly predictive question: *How much worse are the predictions when we destroy the information in one input column on validation data?*

Keep the fitted model *fixed*. Choose feature j; randomly permute only its validation-set values among observations, while leaving the other columns and all target labels unchanged. Denote the modified dataset by $X_{\text{val}}^{\text{perm}(j)}$. For the binary classification Brier loss described below, the source defines

$$\boxed{\text{PI}_j = \text{Brier}\big(y_{\text{val}}, \widehat{\pi}(X_{\text{val}}^{\text{perm}(j)})\big) - \text{Brier}\big(y_{\text{val}}, \widehat{\pi}(X_{\text{val}})\big).} \tag{24}$$

Interpret the sign:

- $\text{PI}_j > 0$: shuffling feature j increased the loss; the model relied on information in that column.
- $\text{PI}_j \approx 0$: little observed change in loss; perhaps the column is unused, or another feature can substitute for it.
- $\text{PI}_j < 0$: this shuffle happened to improve validation loss; investigate variability rather than treating the feature as inherently harmful.

Repeat the permutation with different random shuffles to quantify its Monte Carlo variability; do *not* refit the forest after shuffling.

Correlated predictors require special care. Suppose x_1 and x_2 are very similar measurements of the same phenomenon. Destroying the information in x_1 alone may have little effect because the model can still use x_2. A small individual permutation score therefore need not imply irrelevance of the underlying information. The source proposes jointly permuting (x_1, x_2) *using the same row permutation* to study dependence on the pair, while preserving the relationship between the two columns. Independent shuffles would also destroy their mutual relationship. Even one-column shuffles can create unrealistic combinations of features. Neither MDI nor permutation importance establishes causation; their numbers also have different units and should not be compared as though they were the same quantity.

5.5 Evaluate probabilities separately from decisions

A tree or forest trained for binary classification produces estimated probabilities $\widehat{\pi}_i \in [0, 1]$. There are two fundamentally different questions:

1. Are the *probabilities* quantitatively good estimates?
2. When those estimates are used to make a *yes/no decision*, how expensive are the resulting mistakes?

Brier loss: assessing probabilities. Recall the explicit definition in equations (17) and (18): the Brier loss for one binary outcome is $\ell_{\text{Brier}}(y_i, \widehat{\pi}_i) = (y_i - \widehat{\pi}_i)^2$, and the Brier score is the average of those individual losses. This is a mean squared error applied to probabilities, not to hard labels. Lower is better. If the outcome is $y = 1$, a probability 0.9 contributes $(1 - 0.9)^2 = 0.01$, whereas probability 0.6 contributes $(1 - 0.6)^2 = 0.16$. Both probabilities lead to the same hard prediction at threshold 0.5, but the first receives a better Brier score.

Log loss and extreme leaf probabilities. Log loss for a binary target includes terms

$$-\big[y_i \log \widehat{\pi}_i + (1 - y_i) \log(1 - \widehat{\pi}_i)\big].$$

If $y_i = 1$ but $\widehat{\pi}_i = 0$, the theoretical penalty is infinite; likewise if $y_i = 0$ but $\widehat{\pi}_i = 1$. Pure tree leaves can yield predictions exactly 0 or 1. For *reported* log loss, the companion notebooks clip every candidate's probabilities to the interval

$$\boxed{[10^{-12},\, 1 - 10^{-12}].}$$

The same clipping convention must be applied to all compared candidates.

5.6 Choose a decision threshold from the error costs

Denote by c_{FP} the cost of a false positive (predicting class 1 when the true class is 0), and by c_{FN} the cost of a false negative (predicting 0 when the truth is 1). If correct decisions have zero cost, then for a batch of m observations,

$$\boxed{\text{cost per row} = \frac{c_{\text{FP}}\,\text{FP} + c_{\text{FN}}\,\text{FN}}{m}.} \tag{25}$$

Suppose the estimated probability of class 1 for one new observation is p. If the probability is appropriate for decision-making (in particular, suitably calibrated), then

$$\begin{aligned} \text{expected cost of predicting 1} &= c_{\text{FP}}(1 - p), \\ \text{expected cost of predicting 0} &= c_{\text{FN}} p. \end{aligned}$$

Choose class 1 when its expected cost is no larger:

$$\begin{aligned} c_{\mathrm{FP}}(1-p) &\le c_{\mathrm{FN}}p, \\ c_{\mathrm{FP}} &\le (c_{\mathrm{FP}} + c_{\mathrm{FN}})p, \\ p &\ge \boxed{\frac{c_{\mathrm{FP}}}{c_{\mathrm{FP}} + c_{\mathrm{FN}}}}. \end{aligned}$$

The document's threshold convention predicts class 1 at equality.

The Carseats example in the source. Set $c_{\mathrm{FP}} = 1$ and $c_{\mathrm{FN}} = 3$. A false negative is three times as costly as a false positive. The cost-based threshold is

$$\boxed{t = \frac{1}{1+3} = 0.25.}$$

For example, at estimated positive probability $p = 0.30$:

$$\text{cost if predicting } 1 = 1(0.70) = 0.70, \qquad \text{cost if predicting } 0 = 3(0.30) = 0.90.$$

Hence the lower-cost decision is class 1 even though $p < 0.5$. The companion Carseats notebook also explores a threshold on its validation partition and then freezes it before final testing.

How should decisions be reported? Along with total or per-row cost, report the confusion counts $\mathrm{TP}, \mathrm{FP}, \mathrm{TN}, \mathrm{FN}$ and measures such as

$$\text{precision} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}, \qquad \text{recall} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}},$$

when their denominators are nonzero. Accuracy by itself may mask costly missed positives or an unsuitable decision threshold.

5.7 Three concepts that must remain separate

Quantity	Data used and question answered
MDI importance	Training splits: which features accumulated impurity reductions?
Permutation importance	Validation predictions: which shuffled inputs worsened loss?
OOB score	Internal bootstrap exclusion: how well are held-out training rows predicted?
Final test score	Untouched test data: how well does the frozen model and policy generalize?

Takeaway. OOB evaluation, feature importance, probability scores, and decision costs measure different things. State which observations and which loss produced each reported number. A useful classifier should provide reasonable probabilities, a defensible threshold, and sound evaluation on genuinely held-out data.

6 Source page 6: The three companion notebooks

The final page describes three *independent* notebook-based applications. It is an experiment plan, not a report of numerical test results. Each notebook places its tasks before the code and saves tables and figures after execution. The source specifies **simulation seed 17** and **data-splitting seed 23**. Fixing seeds makes the random experiment reproducible, subject to implementation details.

Notebook	Statistical task	Primary teaching question
A: Service regimes	Regression on synthetic requests	When does a tree fit the true response structure?
B: Carseats	Binary classification on a real dataset	Are the probabilities and resulting decisions sound?
C: Forest diagnostics	Binary classification on synthetic data	How do averaging, feature randomization, OOB assessment and importance interact?

6.1 A. A service with regime changes

`02_A_service_regimes.ipynb`

Generate 1,200 independent requests with payload s, queue length q, and response-time noise ε:

$$s \sim U(0, 10), \qquad q \sim \text{Poisson}(3), \qquad \varepsilon \sim N(0, 8^2). \tag{26}$$

The response (in milliseconds) is

$$\boxed{Y = 40 + 20\mathbf{1}_{\{q>3\}} + 30\mathbf{1}_{\{q>3,\, s>5\}} + \varepsilon.} \tag{27}$$

The indicator $\mathbf{1}_E$ equals 1 when event E occurs, and 0 otherwise. Assuming the noise is independent and centered, the conditional mean is

$$\mathbb{E}[Y \mid s, q] = \begin{cases} 40, & q \le 3, \\ 60, & q > 3,\ s \le 5, \\ 90, & q > 3,\ s > 5. \end{cases} \tag{28}$$

This exactly matches the depth-two tree from (1). The threshold on payload size matters only when the queue length exceeds three: this is a built-in interaction.

Four predictive models.

Method	What it can represent
Mean baseline	A single constant, regardless of the input.
Linear regression	A linear combination of the original features; without engineered threshold terms it cannot represent the sharp regimes exactly.
Decision stump	A single input split and two constant predictions.
Depth-two tree	Two tests along a path; it can represent the true conditional mean exactly by splitting on $q > 3$ then on $s > 5$.

The notebook first verifies a prefix-sum stump implementation against every candidate split in the six-row calculation from page 1, then compares the models' prediction surfaces. It uses *training* cross-validation to select pruning strength and bootstraps training rows to study both the stability of splits and predictions. Finally, it reports frozen-model test errors, complexity and runtime.

What is the irreducible error here? Because the conditional mean (28) is the best squared-error prediction, its theoretical mean squared error is $\mathbb{E}[\varepsilon^2] = 64$ and its population RMSE is

$$\boxed{\text{RMSE}_{\text{ideal}} = 8 \text{ ms.}} \tag{29}$$

This is a consequence of the *specified simulator*, not a measured notebook test score. Finite-sample fitted trees can do worse. The main lesson is that this example favors trees because its conditional mean is inherently piecewise constant; it does *not* imply that trees always outperform linear regression.

6.2 B. Carseats: categorical features and probabilities

`02_B_carseats.ipynb`

Use the provided `Carseats.csv` with 400 rows, and define

$$\boxed{Y = \mathbf{1}_{\{\text{Sales}>8\}}.} \tag{30}$$

The source reports 164 positive observations; this is $164/400 = 0.41$, or 41%. Crucially, `Sales` must *not* appear among the explanatory variables because it defines the target. That would be target leakage, not legitimate prediction.

Categorical predictors. An unordered variable such as shelf location should be handled with one-hot encoding. The encoder must be fitted *inside* each training fold, not on all data before cross-validation; unseen categories need a specified behavior. A tree's leaves then predict class probabilities as the local positive fractions, according to (8).

Probability-first model selection. The notebook compares a logistic-regression baseline with decision trees, selecting depth, minimum leaf size and pruning penalty using training cross-validation with the *Brier score*:

$$\boxed{\text{Brier}(y, \widehat{p}) = \frac{1}{m}\sum_{i=1}^{m}(y_i - \widehat{p}_i)^2.} \tag{31}$$

For one observation, the *Brier loss* is $\ell_{\text{Brier}}(y, p) = (y - p)^2$. Thus selecting by Brier means selecting for *probability prediction* quality, rather than immediate zero-one classification accuracy.

The notebook visualizes a shallow tree, including leaf counts and positive fractions, and asks the reader to translate two paths into human-readable rules on the input features. It checks the validation probabilities and chooses an alarm threshold with cost settings $c_{\text{FP}} = 1$ and $c_{\text{FN}} = 3$. For well-calibrated probabilities and zero cost for correct decisions, the theoretical threshold is

$$\boxed{t = \frac{c_{\text{FP}}}{c_{\text{FP}} + c_{\text{FN}}} = \frac{1}{4}.} \tag{32}$$

The notebook also chooses a threshold from validation data and freezes the resulting decision policy before comparing it with prespecified probability baselines on the final test split.

Main lesson. A tree probability is learned from local counts. Small leaves give unstable probabilities, and a small evaluation sample introduces uncertainty. Probability quality (Brier), hard-classification metrics, and economic decision cost should be kept distinct.

6.3 C. Bagging, decorrelation, OOB and importance

`02_C_forest_diagnostics.ipynb`

The third experiment generates 2,000 rows with 20 input columns. Most features are independent uniform variables on $(-1, 1)$, while x_2 is deliberately made nearly redundant with x_1:

$$x_2 = x_1 + \eta, \qquad \eta \sim N(0, 0.03^2), \tag{33}$$

$$x_4 \sim \text{Bernoulli}(0.5), \tag{34}$$

with x_4 independent. In particular, x_1 and x_2 contain nearly the same information. This is useful for testing how an importance measure handles correlated inputs.

Labels are sampled from a Bernoulli law whose success probability is

$$\boxed{Y \mid x \sim \text{Bernoulli}(\pi(x)), \qquad \pi(x) = \sigma\Big(-0.7 + 2.5x_1 + 2\mathbf{1}_{\{x_3 > 0.2\}}\Big),} \tag{35}$$

where $\sigma(z) = 1/(1 + e^{-z})$ is the logistic function. Unlike the regression simulator in A, the conditional probability in C varies *smoothly* with x_1 but has an additional abrupt regime change when x_3 crosses 0.2.

An illustrative conditional-probability calculation. For a hypothetical input with $x_1 = 0$,

$$\begin{aligned} x_3 \le 0.2: &\quad \pi(x) = \sigma(-0.7) \approx 0.332, \\ x_3 > 0.2: &\quad \pi(x) = \sigma(1.3) \approx 0.786. \end{aligned}$$

These numbers follow from the simulator formula; they are *not* experimental results. Although x_2 correlates strongly with the predictive feature x_1, the generating probability depends on x_1 and x_3, not directly on x_2 or x_4. The remaining noise features have no direct effect in the specified label mechanism.

Experiments.

1. Compare *bagging* against random forests with 300 trees, choosing the feature-subset size m and minimum leaf size by training cross-validation. Bagging uses all features as split candidates; random forests sample a fresh subset at every node.

2. Build bootstrap trees independently to check the *row-specific* OOB averaging formula from (21). Report OOB coverage and examine convergence as the number of trees increases through 600.

3. Distinguish *tree-rule similarity*, measured by how prediction vectors resemble one another across various inputs, from *prediction variability* at a fixed input across repeated training samples. Only the latter corresponds directly to the correlation in the variance formula of page 4.

4. Compare mean decrease in impurity (MDI) and validation permutation importance. Also permute (x_1, x_2) *jointly* with one shared row permutation, preserving their internal correlation while breaking the connection to the target.

The generator uses $\pi(x)$ only to sample Y. The generating probability is *not* an explanatory feature supplied to the classifier: providing it would create a kind of answer leakage.

Main lesson. Averaging can stabilize predictions; feature-subset randomization can reduce similarity among trees; OOB scores measure a particular internal out-of-sample property; and importance scores quantify model reliance in different ways. These need separate diagnostics. The source specifies the experiments, but does not supply the resulting scores or charts.

6.4 How the three applications fit together

	Application A	Application B	Application C
Outcome	Continuous time	Binary high sales	Binary simulated label
Emphasis	Tree structure and stability	Calibration and costs	Forest averaging and diagnostics
Main evaluation	RMSE	Brier and decision cost	CV, OOB and permutation loss

References supplied by the source. The full second lecture is in `PdfLectures`. The accompanying textbook is James et al., *An Introduction to Statistical Learning with Applications in R*, 2nd ed. (2021), Chapters 8.1, 8.2.1 and 8.2.2; Chapter 5 provides bootstrap and model-evaluation background. These are pointers supplied by the original handout, not additional material reviewed here.

Final takeaway. The three notebooks progress from matching tree structure to a regression mechanism (A), to probability-aware classification and cost-sensitive decisions (B), to the distinct diagnostic questions raised by ensemble methods (C).